

**AGH**

**Akademia Górniczo-Hutnicza  
im. Stanisława Staszica w Krakowie**

Wydział Elektrotechniki, Automatyki, Informatyki  
i Inżynierii Biomedycznej (EAIiB)



**ESPHERA**

**ESPhera**

Pełna Dokumentacja Projektu

Wersja 1.0

**Autorzy:** Kacper Wojciuch, Kevin Stuka  
Igor Wadas, Jakub Łabuz

**Opiekun projektu:** dr Dariusz Pałka

Kraków, Polska 2026

# Spis treści

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp</b>                                               | <b>3</b>  |
| 1.1      | Cel i zakres projektu . . . . .                            | 3         |
| 1.2      | Motywacja . . . . .                                        | 3         |
| 1.3      | Główne funkcjonalności . . . . .                           | 4         |
| <b>2</b> | <b>Technologie i narzędzia</b>                             | <b>6</b>  |
| 2.1      | Mikrokontroler ESP32 . . . . .                             | 6         |
| 2.2      | Protokoły komunikacyjne i Architektura Hybrydowa . . . . . | 7         |
| 2.2.1    | Płaszczyzna Sterowania: MQTT . . . . .                     | 7         |
| 2.2.2    | Płaszczyzna Danych: Niestandardowy TCP . . . . .           | 7         |
| 2.3      | Przetwarzanie i transmisja dźwięku (I2S) . . . . .         | 8         |
| 2.4      | Magistrala I2C i Wyświetlacz OLED . . . . .                | 8         |
| 2.4.1    | Autorski sterownik ekranu . . . . .                        | 8         |
| 2.5      | Wykorzystane modele Sztucznej Inteligencji . . . . .       | 9         |
| 2.5.1    | Transkrypcja mowy (STT - Speech to Text) . . . . .         | 9         |
| 2.5.2    | Model językowy (LLM) . . . . .                             | 9         |
| 2.5.3    | Synteza mowy (TTS - Text to Speech) . . . . .              | 9         |
| 2.6      | Bezpieczeństwo i Kryptografia . . . . .                    | 9         |
| 2.6.1    | Biblioteka mbedtls . . . . .                               | 10        |
| 2.6.2    | Algorytm AES-256 GCM . . . . .                             | 10        |
| <b>3</b> | <b>Architektura Systemu</b>                                | <b>11</b> |
| 3.1      | Schemat ogólny . . . . .                                   | 11        |
| 3.1.1    | Maszyna Stanów (State Machine) . . . . .                   | 11        |
| 3.2      | Przepływ danych (Data Flow) . . . . .                      | 13        |
| <b>4</b> | <b>Implementacja Sprzętowa</b>                             | <b>14</b> |
| 4.1      | Wykaz elementów . . . . .                                  | 14        |
| 4.2      | Schemat połączeń . . . . .                                 | 14        |
| 4.3      | Opis podsystemów . . . . .                                 | 15        |
| 4.3.1    | Podsystem Audio (Interfejs I2S) . . . . .                  | 15        |
| 4.3.2    | Interfejs Użytkownika (UI) . . . . .                       | 15        |
| 4.3.3    | Zasilanie . . . . .                                        | 16        |
| <b>5</b> | <b>Implementacja Oprogramowania Wbudowanego (Firmware)</b> | <b>17</b> |
| 5.1      | Struktura projektu i Moduły C++ . . . . .                  | 17        |
| 5.2      | Główna logika aplikacji (esphera.app) . . . . .            | 18        |
| 5.2.1    | Maszyna Stanów (State Machine) . . . . .                   | 18        |
| 5.2.2    | Zarządzanie sesją TCP . . . . .                            | 18        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| 5.3      | Implementacja kluczowych podsystemów . . . . .                            | 19        |
| 5.3.1    | Bezpieczny Provisioning (esphera.ble) . . . . .                           | 19        |
| 5.3.2    | Warstwa transportowa i szyfrowany tunel (esphera.tcp) . . . . .           | 19        |
| 5.3.3    | Abstrakcja WiFi (esphera.wifi) . . . . .                                  | 20        |
| 5.3.4    | Generyczna obsługa dotyku (esphera.touch) . . . . .                       | 20        |
| 5.3.5    | Akwizycja i przetwarzanie dźwięku (esphera.microphone) . . . . .          | 20        |
| 5.3.6    | Odtwarzanie dźwięku i sterowanie wzmacniaczem (esphera.speaker) . . . . . | 21        |
| 5.3.7    | Kryptografia i bezpieczeństwo danych (esphera.aes) . . . . .              | 21        |
| 5.3.8    | Warstwa sterowania i konfiguracji (esphera.mqtt) . . . . .                | 22        |
| 5.3.9    | Obsługa wyświetlacza graficznego (esphera.screen) . . . . .               | 22        |
| 5.3.10   | Wizualizacja stanu urządzenia (esphera.led) . . . . .                     | 22        |
| 5.3.11   | Trwałe przechowywanie danych (esphera.nvs) . . . . .                      | 23        |
| 5.3.12   | Warstwa Prezentacji i Koordynacja UI (esphera.presentation) . . . . .     | 23        |
| 5.3.13   | Zarządzanie energią . . . . .                                             | 24        |
| <b>6</b> | <b>Implementacja Serwera (Backend)</b>                                    | <b>25</b> |
| 6.1      | Struktura aplikacji i środowisko . . . . .                                | 25        |
| 6.2      | Obsługa Bazy Danych . . . . .                                             | 26        |
| 6.3      | Architektura Sieciowa i Protokoły . . . . .                               | 26        |
| 6.3.1    | Data Plane: Serwer TCP (AsyncIO) . . . . .                                | 26        |
| 6.3.2    | Control Plane: Serwis MQTT . . . . .                                      | 27        |
| 6.3.3    | Web Serwer i API (FastAPI) . . . . .                                      | 27        |
| 6.4      | Potok przetwarzania AI (AI Pipeline) . . . . .                            | 27        |
| 6.5      | Bezpieczeństwo danych . . . . .                                           | 28        |
| <b>7</b> | <b>Implementacja Aplikacji Mobilnej</b>                                   | <b>29</b> |
| 7.1      | Technologie i struktura projektu . . . . .                                | 29        |
| 7.2      | Proces parowania (Provisioning BLE) . . . . .                             | 29        |
| 7.3      | Konfiguracja parametrów Asystenta . . . . .                               | 30        |
| 7.4      | Interfejs Użytkownika (UI) . . . . .                                      | 30        |
| <b>8</b> | <b>Podsumowanie i wnioski</b>                                             | <b>31</b> |
| 8.1      | Stopień realizacji celu . . . . .                                         | 31        |
| 8.2      | Wnioski końcowe . . . . .                                                 | 31        |

# Rozdział 1

## Wstęp

Niniejsza praca poświęcona jest projektowi oraz implementacji systemu inteligentnego asystenta głosowego o nazwie „ESPhera”. Projekt łączy w sobie zagadnienia z zakresu systemów wbudowanych (ang. *embedded systems*), Internetu Rzeczy (IoT), komunikacji sieciowej oraz wykorzystania nowoczesnych modeli sztucznej inteligencji (AI) do przetwarzania języka naturalnego.

### 1.1 Cel i zakres projektu

Głównym celem pracy było stworzenie kompletnego, dwukierunkowego interfejsu głosowego, opartego na mikrokontrolerze z rodziny ESP32, który umożliwia prowadzenie płynnej konwersacji z użytkownikiem w czasie rzeczywistym.

Zakres projektu obejmuje dwie główne warstwy:

- **Warstwa sprzętowa (Hardware):** Oparta na układzie SoC ESP32, wyposażona w cyfrowy mikrofon MEMS (interfejs I2S), wzmacniacz audio klasy D, interfejs dotykowy oraz wizualizację stanu urządzenia za pomocą diod LED RGB.
- **Warstwa programowa (Software):**
  - **Firmware:** Oprogramowanie wbudowane napisane w języku C++ z wykorzystaniem frameworka ESP-IDF, realizujące obsługę peryferiów, stosu sieciowego oraz strumieniowanie audio.
  - **Backend:** Serwer napisany w języku Python (FastAPI/AsyncIO), odpowiedzialny za logikę biznesową, szyfrowanie transmisji, zarządzanie bazą danych oraz integrację z zewnętrznymi API sztucznej inteligencji (OpenAI, ElevenLabs).

Istotnym elementem zakresu pracy była implementacja bezpiecznej komunikacji oraz mechanizmu konfiguracji urządzenia (tzw. *provisioning*) z wykorzystaniem technologii Bluetooth Low Energy (BLE).

### 1.2 Motywacja

Główną motywacją do realizacji niniejszego projektu jest aspekt edukacyjny i chęć pogłębionego zrozumienia kluczowych technologii współczesnych systemów asystentów głosowych.

Projekt stanowi doskonałą okazję do praktycznego połączenia kilku poziomów programowania i dziedzin informatyki:

- niskopoziomowe programowanie mikrokontrolerów (obsługa interfejsu I2S, DMA, przerwań, optymalizacja zużycia zasobów),
- programowanie wbudowane z uwzględnieniem ograniczeń sprzętowych,
- projektowanie i implementacja asynchronicznego oprogramowania po stronie serwera,
- integracja z nowoczesnymi Dużymi Modelami Językowymi (LLM),
- przesyłanie i przetwarzanie strumieni audio w czasie zbliżonym do rzeczywistego,
- budowanie kompletnego, end-to-end systemu rozproszonego.

Praca nad takim systemem pozwala na zrozumienie, jak poszczególne warstwy technologiczne współpracują ze sobą, jakie kompromisy trzeba podejmować oraz jakie wyzwania pojawiają się na styku sprzętu, oprogramowania wbudowanego i chmury / serwerów z zaawansowanym przetwarzaniem języka naturalnego.

Projekt ma więc charakter przede wszystkim edukacyjny i eksploracyjny – jego celem jest zdobycie praktycznej wiedzy i umiejętności w zakresie nowoczesnych systemów głosowych typu open-source.

## 1.3 Główne funkcjonalności

Zrealizowany system „ESPhera” oferuje następujące funkcjonalności:

1. **Interakcja głosowa:** Możliwość zadawania pytań i otrzymywania odpowiedzi głosowych w języku naturalnym. System wykorzystuje model *Whisper* do transkrypcji mowy (STT) oraz *ElevenLabs* do syntezy mowy.
2. **Inteligentna konwersacja:** Wykorzystanie modelu GPT (Generative Pre-trained Transformer) pozwala na prowadzenie logicznych i kontekstowych rozmów, wykraczających poza proste komendy sterujące.
3. **Bezpieczna komunikacja:**
  - Szyfrowanie strumienia audio algorytmem AES w celu ochrony prywatności rozmów.
  - Wykorzystanie protokołu MQTT do sterowania asynchronicznego oraz TCP do transmisji danych audio.
4. **Konfiguracja bezprzewodowa (Provisioning):** Mechanizm parowania urządzenia z siecią WiFi za pośrednictwem aplikacji mobilnej i Bluetooth Low Energy (GATT Server), eliminujący konieczność wpisywania danych sieciowych „na sztywno” w kodzie źródłowym.
5. **Interfejs Użytkownika:**

- **Sygnalizacja LED:** Pierścień diod WS2812B informuje użytkownika o aktualnym stanie urządzenia (oczekiwanie, nagrywanie, przetwarzanie, błąd sieci).
  - **Sterowanie dotykowe:** Obsługa przycisków pojemnościowych do inicjowania rozmowy oraz resetowania ustawień sieciowych.
6. **Panel administracyjny:** Dostęp do logów systemowych oraz zarządzanie sparowanymi urządzeniami poprzez dedykowany interfejs WWW i bazę danych SQLite.

# Rozdział 2

## Technologie i narzędzia

Realizacja projektu o tak złożonej architekturze wymagała dobrania odpowiednich komponentów sprzętowych oraz protokołów komunikacyjnych, które zapewniają niskie opóźnienia i niezawodność. W niniejszym rozdziale przedstawiono charakterystykę wykorzystanych technologii oraz uzasadnienie ich wyboru.

### 2.1 Mikrokontroler ESP32

Głównym elementem warstwy sprzętowej urządzenia jest układ SoC (System on Chip) **ESP32**, produkowany przez firmę Espressif Systems. Wybór tego mikrokontrolera podyktowany był jego wysoką wydajnością oraz zintegrowanymi modułami komunikacji bezprzewodowej.

Do najważniejszych cech ESP32, kluczowych z punktu widzenia projektu „ESPhera”, należą:

- **Architektura dwurdzeniowa (Dual-Core):** Układ wyposażony jest w dwa 32-bitowe rdzenie Xtensa LX6 taktowane zegarem do 240 MHz. Pozwala to na rozdelenie zadań: jeden rdzeń (PRO\_CPU) może obsługiwać stos WiFi i Bluetooth, podczas gdy drugi (APP\_CPU) zajmuje się przetwarzaniem audio, obsługą diod LED i logiką aplikacji, co jest krytyczne dla zachowania płynności dźwięku.
- **Wbudowane WiFi i Bluetooth (BLE):** Fabryczna obsługa standardu 802.11 b/g/n oraz Bluetooth Low Energy 4.2 umożliwiła realizację zarówno szybkiej transmisji danych do chmury, jak i energooszczędnego mechanizmu parowania urządzenia (tzw. provisioning).
- **Interfejs I2S (Inter-IC Sound):** Sprzętowe wsparcie dla magistrali I2S umożliwia bezpośrednią komunikację z cyfrowymi mikrofonami MEMS oraz przetwornikami DAC bez obciążania procesora programowym bit-bangingiem.
- **Pamięć:** 520 KB pamięci SRAM oraz obsługa zewnętrznej pamięci Flash (w użytym module 4 MB) pozwala na buforowanie fragmentów strumienia audio.

W projekcie wykorzystano moduł deweloperski **ESP32 DevKit V1**, który udostępnia wprowadzenia GPIO na listwach kołkowych, co ułatwia prototypowanie.

## 2.2 Protokoły komunikacyjne i Architektura Hybrydowa

W celu optymalizacji wydajności i bezpieczeństwa, system „ESPhera” wykorzystuje architekturę hybrydową, rozdzielającą ruch sieciowy na dwie niezależne płaszczyzny: płaszczyznę sterowania (*Control Plane*) oraz płaszczyznę danych (*Data Plane*).

### 2.2.1 Płaszczyzna Sterowania: MQTT

Do zadań administracyjnych, które nie wymagają transmisji w czasie rzeczywistym, wykorzystano protokół MQTT. Pełni on rolę kanału sygnalizacyjnego typu *Low Bandwidth*.

Role protokołu MQTT w systemie:

1. **Provisioning:** Bezpieczna wymiana kluczy kryptograficznych podczas pierwszego uruchomienia.
2. **Konfiguracja (Config):** Zdalna aktualizacja parametrów urządzenia (np. jasność LED, timeout uśpienia) bez konieczności resetowania połączenia audio.

**Bezpieczeństwo transmisji:** W przeciwieństwie do typowych rozwiązań IoT przesyłających dane w formacie tekstowym (JSON), w projekcie zdecydowano się na pełne szyfrowanie ładunku (Payload). Każda wiadomość MQTT jest binarnym blokiem zaszyfrowanym algorytmem AES-256 GCM. Uniemożliwia to podejrzenie konfiguracji lub przejęcie urządzenia nawet w przypadku podsłuchania transmisji na brokerze.

Tabela 2.1: Struktura tematów MQTT (Wszystkie payloady są szyfrowane AES-GCM)

| Temat (Topic)                 | Kierunek     | Opis zdarzenia                 |
|-------------------------------|--------------|--------------------------------|
| devices/provisioning          | ESP → Broker | Próba parowania (Handshake)    |
| devices/provisioning/response | Broker → ESP | Potwierdzenie i nadanie ID     |
| devices/{id}/ask_config       | ESP → Broker | Żądanie aktualnej konfiguracji |
| devices/{id}/config           | Broker → ESP | Binarna struktura konfiguracji |

### 2.2.2 Płaszczyzna Danych: Niestandardowy TCP

Do transmisji głosu (streaming audio) zastosowano bezpośrednie połączenie gniazdowe (Raw TCP Sockets). Jest to dedykowany tunel, otwierany dynamicznie tylko na czas trwania interakcji głosowej.

Specyfika implementacji:

- **Protokół Ramkowy (TLV):** Aby uniknąć problemów z fragmentacją strumienia, zaimplementowano własny protokół warstwy aplikacji. Każdy pakiet danych poprzedzony jest 4-bajtowym nagłówkiem określającym długość następującej po nim zaszyfrowanej porcji audio (Type-Length-Value).
- **Brak narzutu:** Rezygnacja z HTTP/WebSocket na rzecz czystego TCP eliminuje zbędne nagłówki tekstowe, co jest kluczowe przy przesyłaniu próbek dźwięku 24-bit/16kHz w czasie rzeczywistym.



- **Połączenie Trwałe (Persistent Connection):** Połączenie jest utrzymywane przez cały cykl: *Nagrywanie* → *Przetwarzanie AI* → *Odtwarzanie*. Gwarantuje to minimalne opóźnienia (latency), ponieważ odpowiedź serwera może zostać wysłana natychmiastowo tym samym kanałem, bez konieczności ponownego nawiązywania połączenia (Handshake).

## 2.3 Przetwarzanie i transmisja dźwięku (I2S)

W projekcie „ESPhera” zastosowano cyfrowy interfejs audio I2S (Inter-IC Sound). Jest to standard szeregowej komunikacji, zaprojektowany specjalnie do łączenia cyfrowych urządzeń audio.

Kluczowe aspekty implementacji I2S w projekcie:

- **Tryb Master:** Mikrokontroler ESP32 pełni rolę urządzenia nadrzędnego (Master), generując sygnały zegarowe: *BCLK* (Bit Clock) oraz *WS* (Word Select / LR Clock), które synchronizują pracę mikrofonu i wzmacniacza.
- **Wykorzystanie DMA (Direct Memory Access):** Aby odciążyć procesor, transmisja danych między pamięcią RAM a interfejsem I2S odbywa się za pomocą kontrolera DMA. Pozwala to na ciągłe nagrywanie i odtwarzanie dźwięku w tle, podczas gdy główny rdzeń procesora zajmuje się szyfrowaniem danych i obsługą sieci.
- **Standard próbkowania:** Przyjęto parametry dźwięku na poziomie 16 kHz (częstotliwość próbkowania) oraz 16-bitową głębię (rozdzielczość), co zapewnia jakość dźwięku zbliżoną do standardu CD (Compact Disc), wystarczającą dla algorytmów rozpoznawania mowy.

## 2.4 Magistrala I2C i Wyświetlacz OLED

W celu zapewnienia interakcji wizualnej z użytkownikiem, system wyposażono w wyświetlacz graficzny oparty na sterowniku SSD1306 o rozdzielczości 128x64 pikseli. Komunikacja z układem odbywa się za pośrednictwem magistrali I2C (Inter-Integrated Circuit).

### 2.4.1 Autorski sterownik ekranu

W ramach realizacji wymagań projektowych oraz w celu optymalizacji kodu, zrezygnowano z wykorzystania gotowych bibliotek zewnętrznych na rzecz własnej implementacji sterownika sprzętowego. Kod został napisany w języku C++, wykorzystując najnowsze API sterownika I2C dostępne w środowisku ESP-IDF 5.x (`driver/i2c_master.h`).

Proces obsługi wyświetlacza zrealizowano na niskim poziomie abstrakcji, co wymagało szczegółowej analizy dokumentacji technicznej układu SSD1306. Kluczowe elementy implementacji obejmują:

- **Inicjalizacja rejestrów:** Ręczne wysłanie sekwencji komend sterujących (m.in. włączenie pompy ładunkowej `0x8D 0x14`, ustawienie trybu adresowania pamięci `0x20 0x02`).
- **Zarządzanie buforem:** Implementacja bufora ekranu podzielonego na 8 stron (Pages), co wynika ze specyficznej budowy pamięci GDDRAM sterownika SSD1306.

- **Rendering tekstu:** Funkcja `display_text` realizująca mapowanie bitowe znaków z tablicy czcionek (`font8x8`) bezpośrednio na bajty sterujące segmentami wyświetlacza. Algorytm obsługuje zawijanie znaków między stronami pamięci.
- **Manipulacja pikselami:** Funkcja `display_pixel` pozwalająca na zapalanie i gaszenie pojedynczych punktów przy użyciu operacji bitowych na buforze, bez naruszania stanu sąsiednich pikseli.

Takie podejście pozwoliło na pełną kontrolę nad przesyłanymi danymi oraz zredukowało narzut pamięciowy związany z dołączaniem obszernych, uniwersalnych bibliotek graficznych.

## 2.5 Wykorzystane modele Sztucznej Inteligencji

Logika biznesowa asystenta opiera się na chmurze obliczeniowej i wykorzystaniu zewnętrznych API. Ze względu na ograniczoną moc obliczeniową ESP32, mikrokontroler pełni jedynie rolę terminala wejścia-wyjścia, a właściwe przetwarzanie odbywa się na serwerze backendowym w języku Python.

System wykorzystuje potok przetwarzania (pipeline) składający się z trzech etapów:

### 2.5.1 Transkrypcja mowy (STT - Speech to Text)

Do zamiany nagrania głosowego na tekst wykorzystano model OpenAI Whisper. Jest to model typu ASR (Automatic Speech Recognition) wytrenowany na 680 tysiącach godzin danych wielojęzycznych. W projekcie wybrano ten model ze względu na jego wysoką skuteczność w rozpoznawaniu języka polskiego, nawet w obecności szumów tła.

### 2.5.2 Model językowy (LLM)

Generowanie odpowiedzi realizowane jest przez model GPT-4o-nano (Generative Pre-trained Transformer). Odpowiada on za „inteligencję” asystenta – rozumienie kontekstu rozmowy, intencji użytkownika oraz generowanie naturalnie brzmiących odpowiedzi. Dzięki zastosowaniu tzw. *System Prompt*, osobowość asystenta oraz sposób jego wypowiedzi zostały spersonalizowane pod kątem projektu.

### 2.5.3 Synteza mowy (TTS - Text to Speech)

**ElevenLabs:** Wykorzystywany jako główny silnik syntezy. Oferuje on najwyższą dostępną na rynku jakość i naturalność głosu, umożliwiając klonowanie barwy i intonacji.

## 2.6 Bezpieczeństwo i Kryptografia

Urządzenia IoT, w szczególności te wyposażone w mikrofony, wymagają szczególnej dbałości o prywatność przesyłanych danych. W projekcie „ESPhera” zaimplementowano zaawansowaną warstwę kryptograficzną.

### 2.6.1 Biblioteka mbedTLS

Na platformie ESP32 wykorzystano sprzętowo akcelerowaną bibliotekę mbedTLS, która jest standardem w ekosystemie ESP-IDF. Umożliwia ona szybkie szyfrowanie i deszyfrowanie bloków danych bez znaczącego wpływu na opóźnienia transmisji.

### 2.6.2 Algorytm AES-256 GCM

Strumień audio przesyłany przez gniazda TCP jest w całości szyfrowany symetrycznym algorytmem AES (Advanced Encryption Standard) z kluczem o długości 256 bitów.

W projekcie zastosowano tryb GCM (Galois/Counter Mode).

- **Poufność:** Dane są szyfrowane kluczem 256-bitowym, co stanowi obecnie standard przemysłowy.
- **Integralność:** Tryb GCM generuje dodatkowy znacznik uwierzytelniający (Authentication Tag). Dzięki temu system jest odporny na ataki polegające na modyfikacji przesyłanych danych w locie (Tampering).

Serwer backendowy i mikrokontroler współdzielą klucz sesji, który jest bezpiecznie negocjowany podczas procesu parowania (Provisioning) z wykorzystaniem Bluetooth Low Energy.

# Rozdział 3

## Architektura Systemu

System „ESPhera” został zaprojektowany w architekturze klient-serwer, w której rolę klienta pełni urządzenie wbudowane (ESP32), a rolę serwera aplikacja uruchomiona na komputerze lokalnym lub dedykowanym serwerze brzegowym. Takie podejście pozwala na przeniesienie ciężkich obliczeń (transkrypcja, wnioskowanie AI) poza mikrokontroler.

### 3.1 Schemat ogólny

Architektura systemu składa się z trzech głównych warstw logicznych:

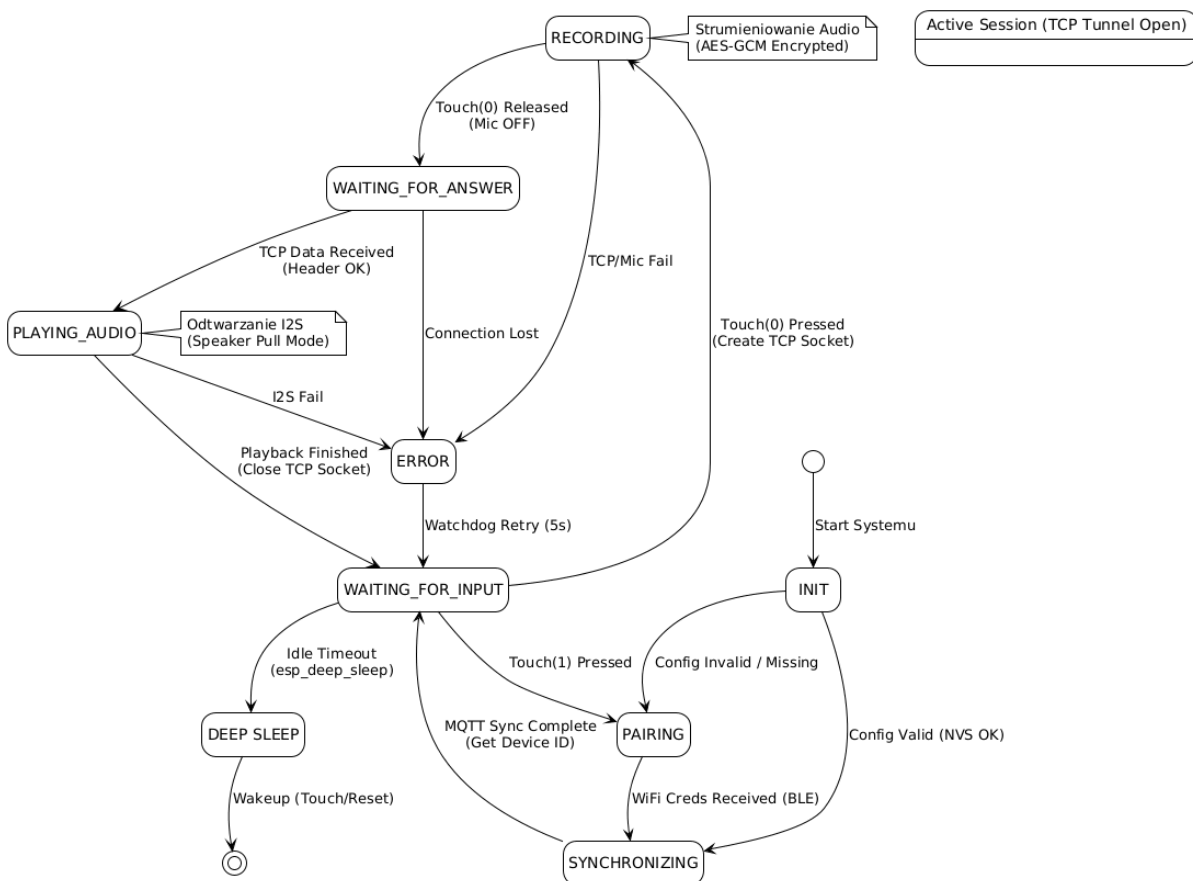
1. **Warstwa Urządzenia (IoT):** Odpowiada za interakcję fizyczną z użytkownikiem. Przechwytuje dźwięk, odtwarza odpowiedzi, steruje oświetleniem LED oraz zarządza połączeniem sieciowym. Oprogramowanie tej warstwy (Firmware) działa w czasie rzeczywistym.
2. **Warstwa Backendowa (Edge Server):** Centralny punkt systemu. Odpowiada za orkiestrację procesów, szyfrowanie/deszyfrowanie strumieni audio (AES-GCM), zarządzanie bazą danych użytkowników oraz komunikację z chmurą.
3. **Warstwa Chmurowa (Cloud AI):** Zewnętrzne serwisy API (OpenAI, Eleven-Labs), które wykonują zadania kognitywne: rozpoznawanie mowy (STT), generowanie tekstu (LLM) oraz syntezę mowy (TTS).
4. **Warstwa Mobilna (Client App):** Pełni rolę interfejsu konfiguracyjnego dla użytkownika końcowego. Odpowiada za proces parowania urządzenia z siecią lokalną (Provisioning) z wykorzystaniem Bluetooth Low Energy oraz umożliwia zdalną personalizację parametrów asystenta (np. zmianę głosu, instrukcji systemowej).

#### 3.1.1 Maszyna Stanów (State Machine)

Zdefiniowano następujące stany pracy urządzenia (enum `State`):

- **INIT:** Inicjalizacja sterowników sprzętowych i montaż partycji NVS.
- **PAIRING:** Uruchamiany automatycznie w przypadku braku skonfigurowanej sieci WiFi. Aktywuje moduł BLE i oczekuje na dane uwierzytelniające z aplikacji mobilnej.

- **SYNCHRONIZING**: Stan przejściowy po nawiązaniu połączenia WiFi. Urządzenie synchronizuje czas (NTP) oraz łączy się z brokerem MQTT w celu pobrania konfiguracji i unikalnego identyfikatora urządzenia (`device_id`).
- **WAITING\_FOR\_INPUT**: Stan spoczynku (Idle). Urządzenie jest podłączone do sieci, a procesor przechodzi w tryb oszczędzania energii, wybudzając się jedynie na przetwarzania od paneli dotykowych.
- **RECORDING**: Aktywacja mikrofonu i nawiązanie bezpiecznego połączenia TCP z serwerem. Strumieniowanie dźwięku odbywa się w czasie rzeczywistym.
- **WAITING\_FOR\_ANSWER**: Po zakończeniu nagrywania (puszczenie przycisku), urządzenie utrzymuje otwarte gniazdo TCP i oczekuje na pierwszy bajt odpowiedzi, nie blokując wątku UI.
- **PLAYING\_AUDIO**: Odbiór zaszyfrowanego strumienia zwrotnego, buforowanie i odtwarzanie dźwięku przez interfejs I2S.
- **ERROR**: Stan awaryjny (np. zerwanie połączenia), z którego system próbuje automatycznie powrócić do pracy po określonym czasie (`Watchdog`).



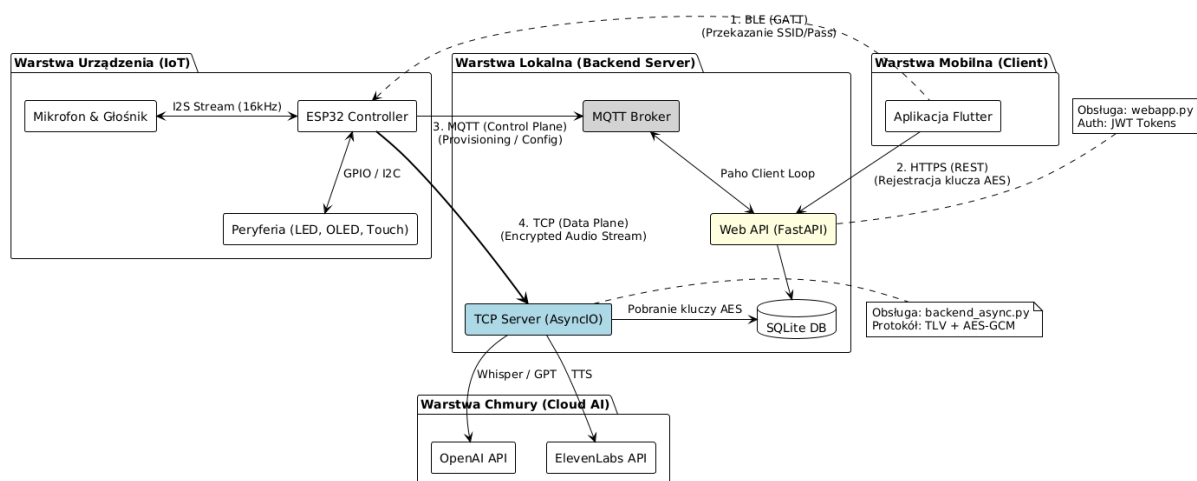
Rysunek 3.1: Diagram Stanów

## 3.2 Przepływ danych (Data Flow)

Proces obsługi pojedynczego zapytania użytkownika przebiega według zoptymalizowanego scenariusza, minimalizującego opóźnienia sieciowe:

1. **Inicjacja:** Po wykryciu dotyku (moduł `esphera.touch`), urządzenie pomija etap sygnalizacji MQTT i natychmiast nawiązuje bezpośrednie połączenie TCP z serwerem (Data Plane).
2. **Autoryzacja w paśmie (In-band Auth):** W ramach pierwszych bajtów strumienia TCP, ESP32 wysyła swój identyfikator oraz zaszyfrowany nagłówek z timestampem. Serwer weryfikuje poprawność klucza AES i czas nadania, nie wymagając odrębnego zapytania o token sesji.
3. **Streaming (Upload):** Urządzenie przesyła zaszyfrowane próbki dźwięku (chunks) w czasie rzeczywistym. Dzięki architekturze strumieniowej, serwer rozpoczyna przetwarzanie (STT) jeszcze przed zakończeniem wypowiedzi użytkownika.
4. **Przetwarzanie AI:** Serwer deszyfruje dane w locie i przekazuje je do potoku AI: Whisper (Transkrypcja) → GPT-4o-mini (Inteligencja) → ElevenLabs (Synteza Głosu).
5. **Streaming (Download):** Odpowiedź audio jest odsyłana do urządzenia tym samym, wciąż otwartym gniazdem TCP. Eliminuje to narzut czasowy związany z ponownym nawiązywaniem połączenia (brak TCP Handshake dla odpowiedzi), co jest kluczowe dla płynności rozmowy.

Taki model komunikacji, w którym MQTT służy jedynie do konfiguracji (w tle), a TCP obsługuje wyłączną transmisję audio, gwarantuje najwyższą możliwą responsywność systemu.



Rysunek 3.2: Schemat architektury systemu ESPhera

# Rozdział 4

## Implementacja Sprzętowa

Warstwa sprzętowa systemu „ESPhera” została zaprojektowana w sposób modułowy, co umożliwia łatwą wymianę komponentów oraz rozbudowę urządzenia w przyszłości. W niniejszym rozdziale przedstawiono wykaz wykorzystanych elementów, schemat połączeń oraz opis poszczególnych podsystemów.

### 4.1 Wykaz elementów

Do budowy urządzenia wykorzystano komponenty ogólnodostępne na rynku (COTS - Commercial Off-The-Shelf), co zapewnia powtarzalność konstrukcji i niski koszt wdrożenia. Zestawienie elementów przedstawiono w Tabeli 4.1.

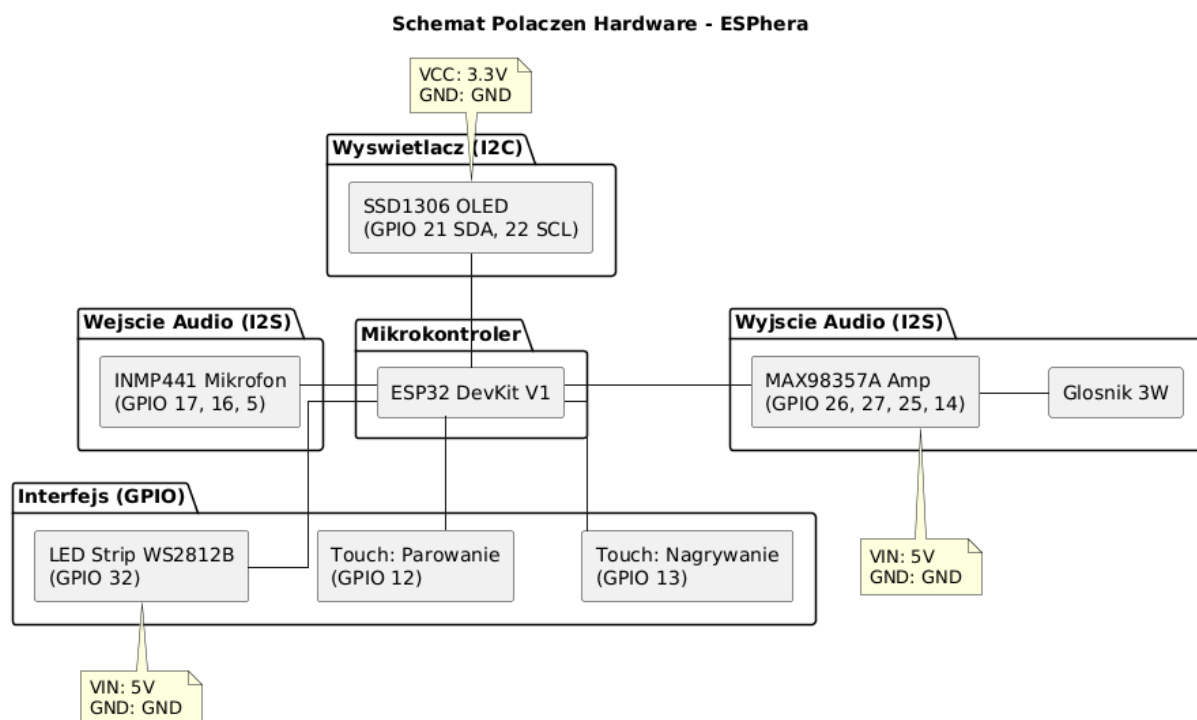
Tabela 4.1: Wykaz komponentów sprzętowych

| Lp. | Nazwa elementu       | Funkcja w systemie                                           |
|-----|----------------------|--------------------------------------------------------------|
| 1   | ESP32 DevKit V1      | Główna jednostka sterująca (SoC)                             |
| 2   | INMP441              | Cyfrowy mikrofon wielokierunkowy (I2S)                       |
| 3   | MAX98357A            | Wzmacniacz audio klasy D z przetwornikiem DAC (I2S)          |
| 4   | Głośnik 3W 4Ω        | Odtwarzanie dźwięku                                          |
| 5   | WS2812B (Ring/Strip) | Pierścień diod LED RGB (Sygnalizacja stanu)                  |
| 6   | SSD1306 OLED         | Wyświetlacz graficzny 0.96” (I2C)                            |
| 7   | Elektrody dotykowe   | Zrealizowane bezpośrednio na przewodach podłączonych do GPIO |

### 4.2 Schemat połączeń

Prawidłowe połączenie magistrali komunikacyjnych (I2S, I2C) jest kluczowe dla stabilności systemu. Szczególną uwagę zwrócono na separację linii zasilania, aby cyfrowe sterowanie diodami LED nie wprowadzało zakłóceń do toru audio.

Schemat blokowy połączeń zrealizowanych w projekcie przedstawiono na Rysunku 4.1.



Rysunek 4.1: Schemat połączeń elektrycznych systemu ESPhera

## 4.3 Opis podsystemów

### 4.3.1 Podsystem Audio (Interfejs I2S)

System audio realizuje dwukierunkową transmisję dźwięku przy użyciu magistrali cyfrowej I2S. Dzięki wyeliminowaniu analogowych sygnałów na odcinku między mikrofonem/wzmacniaczem a procesorem, układ jest wysoce odporny na zakłócenia elektromagnetyczne.

- **Wejście (Mikrofon INMP441):** Mikrofon podłączony jest do pinów: SCK (GPIO 17), WS (GPIO 16) oraz SD (GPIO 5). Pin L/R mikrofonu został zwarty do masy (GND), co konfiguruje go do przesyłania danych w kanale lewym.
- **Wyjście (Wzmacniacz MAX98357A):** Układ przetwarza cyfrowy sygnał audio na analogowy sygnał mocy sterujący głośnikiem. Wykorzystuje piny: BCLK (GPIO 26), LRC (GPIO 27) oraz DIN (GPIO 25). Pin SD (GPIO 14) służy do programowego wyłączania wzmacniacza (Mute) w celu eliminacji trzasków podczas startu systemu.

### 4.3.2 Interfejs Użytkownika (UI)

Interakcja z użytkownikiem odbywa się wielokanałowo, poprzez dotyk, światło oraz (opcjonalnie) obraz.

**1. Sterowanie dotykowe:** Wykorzystano wbudowany w ESP32 sterownik czujników pojemnościowych (Touch Sensor). Zamiast fizycznych przycisków mechanicznych, wprowadzono przewody pełniące rolę elektrod:



- **GPIO 13 (Touch 4):** Przycisk aktywacji nagrywania (Push-to-Talk).
- **GPIO 12 (Touch 5):** Przycisk funkcyjny (Parowanie / Reset ustawień WiFi).

**2. Sygnalizacja świetlna:** Pasek diod adresowalnych WS2812B sterowany jest jednym przewodem sygnałowym (GPIO 32). Każda dioda może świecić niezależnym kolorem, co pozwala na tworzenie złożonych animacji (np. pulsowanie podczas myślenia AI, czy efekt „Knight Rider” przy starcie).

**3. Wyświetlacz OLED:** Układ wyposażono w ekran SSD1306 podłączony do magistrali I2C (SDA: GPIO 21, SCL: GPIO 22). Stanowi on element przygotowany pod przyszłą rozbudowę oprogramowania o wizualizację tekstu transkrypcji.

### 4.3.3 Zasilanie

Cały układ zasilany jest napięciem 5V (poprzez port USB mikrokontrolera).

- **Szyna 5V (VIN):** Zasila bezpośrednio wzmacniacz audio oraz diody LED, które wymagają wyższego napięcia dla uzyskania pełnej mocy.
- **Szyna 3.3V:** Zasila logikę mikrokontrolera, mikrofon, wyświetlacz oraz moduł dotykowy. Napięcie to jest stabilizowane przez regulator LDO wbudowany w płytke DevKit.

## Rozdział 5

# Implementacja Oprogramowania Wbudowanego (Firmware)

Warstwa oprogramowania wbudowanego (Firmware) została zaimplementowana w języku C++20 z wykorzystaniem natywnego SDK producenta mikrokontrolera – ESP-IDF v5.x. Decyzja o porzuceniu języka C na rzecz nowoczesnego C++ podyktowana była chęcią zwiększenia czytelności kodu, bezpieczeństwa typów oraz wykorzystania zaawansowanych mechanizmów abstrakcji, takich jak szablony (templates) czy moduły (C++ Modules).

### 5.1 Struktura projektu i Moduły C++

W przeciwieństwie do klasycznego podejścia opartego na plikach nagłówkowych (.h) i źródłowych (.c/.cpp), w projekcie zastosowano eksperymentalną obsługę modułów C++ (.cppm). Pozwoliło to na hermetyzację logiki poszczególnych podsystemów w wydzielonych przestrzeniach nazw.

Kod źródłowy został podzielony na następujące moduły logiczne w przestrzeni nazw **esphera**:

- **esphera.app**: Główny orkiestrator aplikacji, realizujący wzorzec Singleton i zarządzający maszyną stanów.
- **esphera.wifi**: Abstrakcja nad stosem sieciowym LwIP/ESP-Netif, obsługująca automatyczne łączenie i ponawianie prób.
- **esphera.tcp**: Klient stałego połączenia TCP z obsługą szyfrowania i keep-alive.
- **esphera.ble**: Implementacja serwera GATT do bezpiecznego provisioningu (przekazywania danych WiFi).
- **esphera.mqtt**: Klient protokołu MQTT.
- **esphera.microphone**: Obsługa rejestracji dźwięku przez interfejs I2S, buforowanie oraz konwersja do formatu PCM (24-bit).
- **esphera.speaker**: Obsługa odtwarzania dźwięku (I2S - TX) i sterowanie wzmacniaczem.
- **esphera.aes**: Implementacja kryptografii (AES-GCM) w oparciu o bibliotekę mbed-TLS.

- **esphera.led:** Obsługa wizualizacji świetlnej na diodach WS2812B z wykorzystaniem asynchronicznej kolejki poleceń.
- **esphera.touch:** Generyczna klasa obsługi przycisków dotykowych, wykorzystująca szablony C++.
- **esphera.screen:** Autorski sterownik wyświetlacza OLED z kolejkowaniem poleceń graficznych.
- **esphera.nvs:** Zarządzanie pamięcią nieulotną.
- **esphera.presentation:** Warstwa orkiestracji UI, zarządzająca synchronizacją ekranu OLED i diod LED z obsługą przerywania animacji.

## 5.2 Główna logika aplikacji (esphera.app)

Sercem oprogramowania układowego jest klasa **App**, zaimplementowana jako Singleton. Odpowiada ona za orkiestrację wszystkich podsystemów oraz zarządzanie cyklem życia urządzenia przy użyciu deterministycznej maszyny stanów.

### 5.2.1 Maszyna Stanów (State Machine)

Zdefiniowana w rozdziale o architekturze maszyna stanów została zaimplementowana wprost w metodzie `run()` klasy **App**.

Szczegóły implementacji w C++:

- **Atomowość:** Aktualny stan przechowywany jest w zmiennej `std::atomic<State>`, co zapewnia bezpieczeństwo wątkowe (Thread Safety). Jest to kluczowe, ponieważ zmiana stanu może zostać zainicjowana zarówno przez główną pętlę, jak i przez przerwanie od panelu dotykowego lub callback z wątku sieciowego.
- **Pętla główna:** Logika oparta jest na nieskończonej pętli `while(true)` z instrukcją `switch`, która w każdej iteracji sprawdza aktualny stan i wykonuje przypisane do niego zadania (np. sprawdzenie warunków przejścia).
- **Orkiestracja:** Przejście między stanami (np. z `RECORDING` do `WAITING_FOR_ANSWER`) automatycznie wyzwała odpowiednie metody w podsystemach (np. zatrzymanie mikrofonu, wysłanie komendy do warstwy prezentacji), co gwarantuje spójność całego urządzenia.

### 5.2.2 Zarządzanie sesją TCP

W przeciwieństwie do typowych rozwiązań IoT, które utrzymują stałe połączenie z chmurą, system ESPhera dynamicznie zarządza zasobami sieciowymi. Obiekt TCP jest alokowany na stercie (`new TCP`) w momencie rozpoczęcia nagrywania i niszczone (`delete`) dopiero po zakończeniu odtwarzania odpowiedzi. Gwarantuje to minimalne opóźnienia w trakcie rozmowy (brak handshake'u między pytaniem a odpowiedzią) oraz oszczędność pamięci i energii w stanie czuwania.

## 5.3 Implementacja kluczowych podsystemów

### 5.3.1 Bezpieczny Provisioning (esphera.ble)

Moduł Bluetooth odpowiada za proces parowania urządzenia z siecią WiFi. Zaimplementowano tu niestandardowy profil GATT, zoptymalizowany pod kątem bezpiecznego przesyłania danych konfiguracyjnych.

Kluczowe mechanizmy:

- **Security Handshake:** Aplikacja mobilna musi najpierw przesłać zaszyfrowany klucz sesji na charakterystykę `CUSTOM_KEY`. Jest on weryfikowany przez mikrokontroler przy użyciu zaszytego klucza fabrycznego. Dopiero poprawna weryfikacja odblokowuje możliwość zapisu SSID i hasła.
- **Obsługa Long Writes:** Hasła WiFi mogą przekraczać standardowy rozmiar pakietu BLE (MTU). Moduł implementuje mechanizm *Prepare Write / Execute Write*, pozwalający na składanie długich ciągów danych z mniejszych fragmentów w buforze pamięci RAM przed ich przetworzeniem.
- **Szyfrowanie warstwy aplikacji:** Niezależnie od zabezpieczeń łącza Bluetooth, same dane (SSID, Hasło) są przesyłane w formie zaszyfrowanej algorytmem AES-256 i deszyfrowane programowo dopiero po skompletowaniu całego bufora.

### 5.3.2 Warstwa transportowa i szyfrowany tunel (esphera.tcp)

Moduł ten stanowi wrapper na gniazda (BSD Sockets) z biblioteki LwIP. Zamiast standardowego podejścia HTTP/REST, zaimplementowano tu stałe, szyfrowane połączenie TCP (Persistent Connection), co drastycznie redukuje opóźnienia.

Szczegóły implementacji protokołu:

- **TCP Keep-Alive:** Skonfigurowano opcje gniazda (`SO_KEEPALIVE`, `TCP_KEEPINTVL`), aby system operacyjny automatycznie podtrzymywał połączenie i wykrywał zerwanie łącza bez konieczności ciągłego przesyłania danych aplikacyjnych.
- **Protokół Ramkowy (Framing):** Aby uniknąć fragmentacji strumienia, każda wysyłana wiadomość poprzedzona jest nagłówkiem zawierającym rozmiar zaszyfrowanych danych ('size\_t'). Odbiorca najpierw odczytuje rozmiar, alokuje odpowiedni bufor, a następnie pobiera właściwy payload.
- **Handshake Autoryzacyjny:** Natychmiast po nawiązaniu połączenia TCP, moduł wysyła 'device\_id' oraz zaszyfrowany blok autoryzacyjny (**Auth Header**). Dopiero po weryfikacji tego pakietu przez serwer, tunel uznawany jest za aktywny.
- **Wielowątkowość:** Moduł udostępnia metodę `wait_for_byte`, która uruchamia dedykowane zadanie FreeRTOS oczekujące na dane przychodzące. Pozwala to na asynchroniczną reakcję na komendy serwera bez blokowania głównej pętli programu.

### 5.3.3 Abstrakcja WiFi (esphera.wifi)

Klasa `WiFi` stanowi warstwę abstrakcji (Wrapper) nad natywnym stosem sieciowym `LwIP` oraz systemem zdarzeń `ESP-IDF`, upraszczając zarządzanie połączeniem sieciowym w głównej logice aplikacji.

Zastosowane rozwiązania architektoniczne:

- **Ukrycie Event Loop:** Biblioteka encapsuluje skomplikowaną logikę rejestracji i obsługi zdarzeń systemowych (`WIFI_EVENT`, `IP_EVENT`). Zamiast ręcznej obsługi stanów w głównym pliku, klasa udostępnia czytelny interfejs oparty na callbackach, powiadamiając aplikację tylko o kluczowych zmianach (np. uzyskanie adresu IP).
- **Inteligentne ponawianie:** Zaimplementowano licznik `retry_count` z limitem 3 prób (`WIFI_MAX_RETRIES_NEW`) oraz opóźnieniem 10 sekund między próbami. Jeśli połączenie nie powiedzie się pomimo retansmisji (np. błędne hasło), urządzenie zgłasza status `FAILED`, co pozwala maszynie stanów na powrót do trybu parowania.
- **Bezpieczeństwo wątkowe:** Stan połączenia oraz liczniki prób są chronione przy użyciu typów atomowych (`std::atomic`), co gwarantuje poprawność działania w wielowątkowym środowisku `FreeRTOS`, gdzie przerwania sieciowe mogą współbieżnie modyfikować stan obiektu.

### 5.3.4 Generyczna obsługa dotyku (esphera.touch)

Moduł ten demonstruje wykorzystanie zaawansowanych cech języka `C++` (szablony) do obsługi sprzętowego sensora pojemnościowego wbudowanego w `ESP32`.

Szczegóły implementacji:

- **Metaprogramowanie (Templates):** Klasa `TouchSensor<N>` jest szablonem przyjmującym liczbę elektrod jako parametr kompilacji. Pozwala to na statyczną alokację zasobów i wygenerowanie zoptymalizowanego kodu dla konkretnej konfiguracji sprzętowej (w tym przypadku jednej elektrody), bez narzutu dynamicznej alokacji pamięci.
- **Przetwarzanie w tle (RTOS):** Obiekt sensora automatycznie powołuje dedykowane zadanie `FreeRTOS`. Zadanie to cyklicznie odczytuje wartości z rejestrów sprzętowych, dzięki czemu główny wątek aplikacji nie musi zajmować się pollingiem (aktywnym oczekiwaniem) na dotyk.
- **Analiza sygnału:** Surowe dane z sensora poddawane są filtracji w celu eliminacji szumów elektromagnetycznych. Algorytm analizuje czas trwania sygnału, rozróżniając gesty użytkownika: „krótkie kliknięcie” (aktywacja asystenta) oraz „długie przytrzymanie” (Long Press – reset do ustawień fabrycznych).

### 5.3.5 Akwizycja i przetwarzanie dźwięku (esphera.microphone)

Klasa `Microphone` stanowi abstrakcję nad sterownikiem `I2S` w trybie `RX` (Master), odpowiedzialną za pobieranie surowych danych z mikrofonu `MEMS`.

Kluczowe aspekty implementacji:

- **Strumieniowanie przez Callback:** Zrezygnowano z buforowania całego nagrania w pamięci RAM. Zamiast tego, zadanie nagrywania cyklicznie wywołuje funkcję zwrotną (`SaveCallback`), przekazując małe porcje danych bezpośrednio do warstwy transportowej (TCP). Pozwala to na nagrywanie wypowiedzi o dowolnej długości przy minimalnym zużyciu pamięci.
- **Konwersja 24-bit:** Sterownik I2S pracuje na słowach 32-bitowych (wymóg sprzętowy), jednak moduł w locie dokonuje bitowego przesunięcia i pakowania danych do formatu 24-bitowego. Redukuje to objętość transmisji o 25% bez utraty jakości dźwięku.
- **Metadane WAV:** Ponieważ długość nagrania nie jest znana w momencie startu, moduł zlicza przesłane bajty i po zakończeniu rejestracji generuje poprawny nagłówek RIFF WAVE, który jest wysyłany jako zakończenie strumienia.

### 5.3.6 Odtwarzanie dźwięku i sterowanie wzmacniaczem (`esphera.speaker`)

Moduł obsługuje kanał nadawczy interfejsu I2S (TX) w trybie Master, sterując zewnętrznym przetwornikiem DAC/wzmacniaczem MAX98357A.

Kluczowe rozwiązania sprzętowe i programowe:

- **Architektura typu „Pull”:** Zastosowano wzorzec *Fetch Callback*. Zadanie odtwarzania (`playback_task`) cyklicznie wywołuje funkcję dostarczającą dane, która pobiera zdekodowane próbki PCM bezpośrednio z bufora sieciowego. Eliminuje to potrzebę stosowania dużych buforów pośrednich, oszczędzając pamięć RAM.
- **Sprzętowe wyciszenie (Anti-Pop):** Moduł steruje pinem `GPIO_SD` (Shutdown). Pin ten jest ustawiany w stan wysoki tylko na czas aktywnego odtwarzania i ściągany do masy natychmiast po zakończeniu strumienia. Fizycznie odcina to zasilanie końcówki mocy wzmacniacza, eliminując szumy jałowe i trzaski.
- **Synchronizacja wątków:** Wykorzystano zmienną atomową `std::atomic_bool` `playing` do bezpiecznego zarządzania cyklem życia zadania odtwarzania w środowisku wielowątkowym FreeRTOS.

### 5.3.7 Kryptografia i bezpieczeństwo danych (`esphera.aes`)

Moduł ten stanowi wrapper na bibliotekę `mbedTLS`, implementując nowoczesny standard **AES-256 GCM** zamiast prostych trybów blokowych.

Kluczowe aspekty implementacji:

- **Akceleracja sprzętowa:** Wykorzystanie wbudowanego w ESP32 koprocatora kryptograficznego odciąża główny procesor (CPU). Umożliwia to szyfrowanie strumienia audio w czasie rzeczywistym bez wpływu na płynność nagrywania i transmisji.
- **Integralność i Unikalność:** Tryb GCM dołącza do każdego pakietu znacznik uwierzytelniający (Auth Tag) oraz losowy wektor inicjujący (IV). Chroni to system przed modyfikacją danych w locie oraz ukrywa powtarzające się wzorce (np. fragmenty cisy), uodparniając komunikację na ataki statystyczne.

### 5.3.8 Warstwa sterowania i konfiguracji (esphera.mqtt)

Podczas gdy strumień audio przesyłany jest przez dedykowany tunel TCP, moduł MQTT odpowiada za płaszczyznę sterowania (*Control Plane*) systemu.

Funkcjonalności modułu:

- **Separacja ruchu:** Wykorzystanie lekkiego protokołu MQTT do zadań administracyjnych odciąża główny kanał transmisji audio, zapobiegając zatorom (Head-of-Line Blocking) podczas przesyłania dźwięku.
- **Provisioning i Autoryzacja:** Moduł obsługuje tematy związane z pierwszym uruchomieniem urządzenia. Po nawiązaniu połączenia WiFi, to właśnie przez MQTT następuje wymiana kluczy kryptograficznych i przypisanie urządzenia do użytkownika.
- **Zdalna konfiguracja:** Urządzenie subskrybuje temat konfiguracyjny, co umożliwia serwerowi zdalną zmianę parametrów pracy (np. zmianę promptu systemowego AI lub ustawień timeoutu) bez konieczności aktualizacji całego firmware'u.
- **Szyfrowanie Payloadu:** Mimo że MQTT może działać po TLS, warstwa aplikacji dodatkowo szyfruje treść wiadomości algorytmem AES-GCM (tym samym, co audio), zapewniając poufność danych konfiguracyjnych (End-to-End Encryption).

### 5.3.9 Obsługa wyświetlacza graficznego (esphera.screen)

Moduł ten implementuje niskopoziomową obsługę sterownika SSD1306 (OLED 128x64), rezygnując z gotowych, pamięciożernych bibliotek graficznych na rzecz wydajnego rozwiązania dedykowanego.

Kluczowe aspekty implementacji:

- **Architektura Asynchroniczna:** Transmisja I2C jest operacją stosunkowo wolną i blokującą. Aby zapobiec zamrażaniu głównego wątku aplikacji podczas odświeżania ekranu, zastosowano wzorzec z kolejką komunikatów (`screen_queue`). Komendy takie jak `DisplayText` czy `ClearScreen` są buforowane i przetwarzane sekwencyjnie w dedykowanym zadaniu (`screen_task`).
- **Bezpośrednia kontrola sterownika:** Komunikacja odbywa się poprzez wysyłanie surowych bajtów sterujących (Command Byte 0x00) oraz danych (Data Byte 0x40) przy użyciu nowoczesnego API `driver/i2c_master.h`. Pozwoliło to na precyzyjną konfigurację pompy ładunkowej oraz trybów adresowania pamięci GDDRAM.
- **Renderowanie tekstu:** Zaimplementowano własny algorytm mapowania znaków ASCII na strony pamięci wyświetlacza, z obsługą zawijania wierszy oraz inwersji kolorów (negatyw), co jest wykorzystywane do sygnalizowania aktywności asystenta.

### 5.3.10 Wizualizacja stanu urządzenia (esphera.led)

Moduł ten odpowiada za asynchroniczne sterowanie paskiem diod WS2812B, realizując wzorzec projektowy Producent-Konsument.

Kluczowe aspekty implementacji:

- **Kolejkowanie poleceń (FreeRTOS):** Aby czasochłonne animacje świetlne nie blokowały głównego wątku aplikacji (i np. obsługi WiFi), wszystkie komendy są przesyłane do kolejki (`xQueueSend`) i przetwarzane w tle przez dedykowane zadanie.
- **Sterowanie sprzętowe (RMT):** Wykorzystano peryferium RMT (Remote Control) do generowania precyzyjnych przebiegów sygnałowych. Odciąża to procesor od konieczności programowego sterowania pinami (*bit-banging*), co jest krytyczne przy równoległej pracy z WiFi i Bluetooth.

### 5.3.11 Trwałe przechowywanie danych (`esphera.nvs`)

Moduł odpowiada za interakcję z partycją NVS (Non-Volatile Storage) w pamięci flash, wykorzystując system plików typu klucz-wartość.

Funkcjonalności:

- **Przechowywanie poświadczeń:** W bezpieczny sposób zapisuje SSID, hasło do sieci oraz 256-bitowy klucz AES wymieniony podczas procesu parowania.
- **Serializacja konfiguracji:** Moduł definiuje strukturę `Config` (zawierającą m.in. jasność LED i czas do uśpienia), która jest bezpośrednio mapowana na binarny blob pamięci. Umożliwia to łatwą aktualizację ustawień urządzenia poprzez nadpisanie obszaru pamięci nowym blokiem danych otrzymanym z MQTT.
- **Autonaprawa:** W procesie inicjalizacji zaimplementowano mechanizm detekcji uszkodzeń lub niezgodności wersji partycji. W przypadku błędu (`ESP_ERR_NVS_NEW_VERSION_FOUND`), moduł automatycznie formatuje partycję, przywracając urządzenie do stanu fabrycznego, co zapobiega pętlom restartów (Boot Loop).

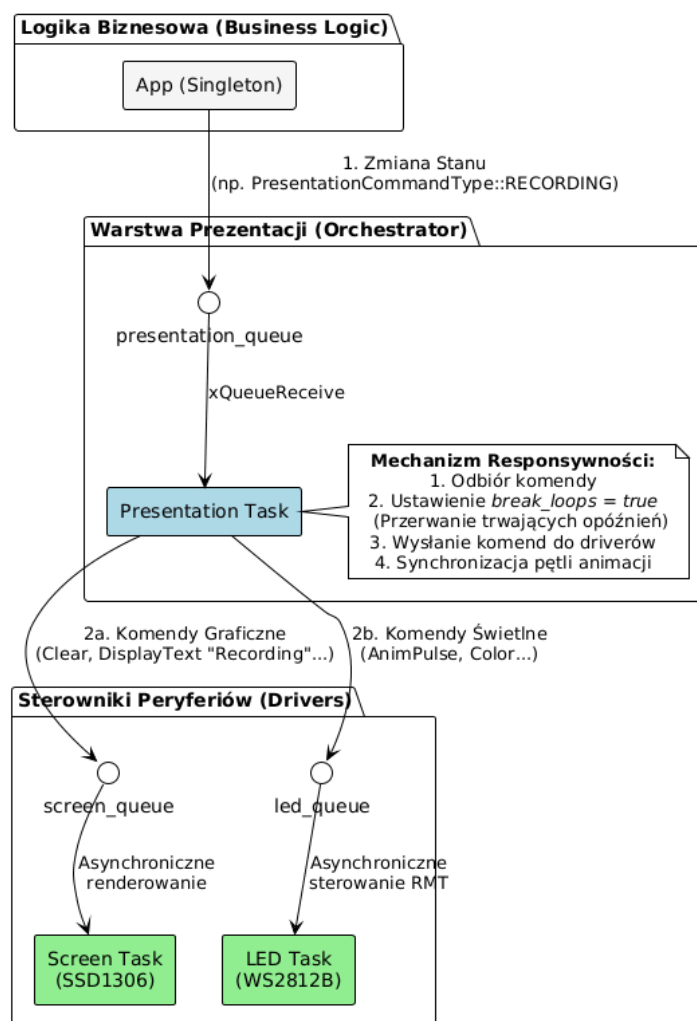
### 5.3.12 Warstwa Prezentacji i Koordynacja UI (`esphera.presentation`)

Moduł ten pełni rolę centralnego orkiestratora interfejsu użytkownika, separując logikę biznesową aplikacji od niskopoziomowego sterowania peryferiami wyjściowymi (Ekran OLED, Pasek LED).

Kluczowe mechanizmy implementacji:

- **Centralizacja Stanów Wizualnych:** Moduł mapuje abstrakcyjne stany systemu (np. `WAITING_FOR_INPUT`, `RECORDING`, `PAIRING`) na konkretne sekwencje wizualne. Dzięki temu główna maszyna stanów nie musi zarządzać szczegółami wyświetlania tekstów czy kolorami diod.
- **Responsywność (Interruptible Animations):** Wiele efektów wizualnych to długotrwałe pętle (np. migający tekst "Waiting..."). Aby zapewnić natychmiastową reakcję urządzenia na zmianę stanu, zaimplementowano mechanizm `break_loops`. Nadejście nowej komendy do kolejki natychmiast ustawia flagę `volatile`, która przerywa trwające opóźnienia (`delay_until_break_ms`), umożliwiając płynne przełączenie kontekstu wizualnego bez odczuwalnych opóźnień.
- **Synchronizacja Mediów:** Moduł gwarantuje, że komunikaty na wyświetlaczu i efekty świetlne są zawsze spójne (np. animacja pulsującego światła jest zsynchronizowana z tekstem "Processing"), co jest trudne do osiągnięcia przy sterowaniu z osobnych wątków.





Rysunek 5.1: Diagram UI Orchestrator

### 5.3.13 Zarządzanie energią

System implementuje zaawansowane mechanizmy oszczędzania energii. W stanie `WAITING_FOR_INPUT` monitorowany jest czas bezczynności. Po przekroczeniu zdefiniowanego w konfiguracji limitu (`sleep_timeout`), urządzenie przechodzi w stan głębokiego uśpienia (`Deep Sleep`). Wybudzenie następuje wyłącznie poprzez fizyczne dotknięcie sensora pojemnościowego lub twardy reset, co pozwala na wielomiesięczną pracę na zasilaniu bateryjnym.

# Rozdział 6

## Implementacja Serwera (Backend)

Warstwa serwerowa (Backend) odpowiada za koordynację komunikacji z urządzeniami IoT, obsługę aplikacji klienckich (Web/Mobile), zarządzanie bazą danych oraz integrację z usługami sztucznej inteligencji.

Serwer został zaimplementowany w języku Python 3.11, wykorzystując nowoczesną architekturę asynchroniczną oraz framework **FastAPI**. Aplikacja działa jako monolit, który w ramach jednej pętli zdarzeń (Event Loop) obsługuje równolegle serwer HTTP, gniazda TCP oraz klienta MQTT.

### 6.1 Struktura aplikacji i środowisko

Główne biblioteki i frameworki wykorzystane w projekcie:

- **fastapi & uvicorn:** Nowoczesny, wysokowydajny framework webowy służący do budowy API REST oraz panelu administracyjnego. Uvicorn pełni rolę serwera aplikacji (ASGI).
- **asyncio:** Biblioteka standardowa Pythona do obsługi współbieżności. Jest fundamentem obsługi niestandardowego serwera TCP (Data Plane) oraz zadań w tle.
- **paho-mqtt:** Klient protokołu MQTT działający w trybie nieblokującym, odpowiedzialny za warstwę sterowania (Control Plane).
- **cryptography & pyjwt:** Biblioteki odpowiedzialne za bezpieczeństwo: implementację szyfrowania AES-GCM (zgodną z mbedTLS na ESP32) oraz obsługę tokenów JWT do autoryzacji użytkowników.
- **openai & elevenlabs:** Oficjalne biblioteki SDK do komunikacji z chmurowymi modelami AI.
- **pydub:** Biblioteka do manipulacji plikami audio (konwersja, resampling), wykorzystująca w tle narzędzie FFmpeg.
- **sqlite3:** Obsługa relacyjnej bazy danych z mechanizmem blokad wątkowych (`threading.Lock`).

## 6.2 Obsługa Bazy Danych

System wykorzystuje relacyjną bazę danych SQLite (plik `esphera_pro.db`). Ze względu na to, że dostęp do danych następuje z wielu kontekstów jednocześnie (Wątek MQTT, Handlerzy TCP, Endpointy HTTP), wszystkie operacje zapisu i odczytu są synchronizowane globalnym obiektem blokady (`db_lock`). Zapobiega to uszkodzeniu pliku bazy danych i błędom współbieżności.

Struktura danych obejmuje cztery główne tabele:

1. **users:** Przechowuje dane uwierzytelniające właścicieli urządzeń (adres email oraz bezpieczny hash hasła bcrypt).
2. **devices:** Rejestr urządzeń fizycznych. Kluczowymi polami są `aes_key` (unikalny klucz sesji) oraz `status` (określający, czy urządzenie jest w trakcie parowania 'pending', czy już aktywne 'paired').
3. **config:** Przechowuje spersonalizowaną konfigurację, która jest dynamicznie wstrzykiwana do procesów AI i sterowania sprzętem:
  - `system_prompt`: Instrukcja definiująca osobowość asystenta.
  - `sound_model_id`: Identyfikator głosu syntezy ElevenLabs.
  - `led_intensity` i `sleep_timeout`: Parametry sprzętowe przesyłane do ESP32.
4. **logs:** Tabela audytowa rejestrująca historię zapytań głosowych (transkrypcja IN) oraz odpowiedzi asystenta (tekst OUT).

## 6.3 Architektura Sieciowa i Protokoły

Backend implementuje architekturę hybrydową, obsługując trzy różne protokoły sieciowe w ramach jednej instancji aplikacji. Start systemu (`@app.on_event("startup")`) inicjuje w tle serwery TCP i MQTT, podczas gdy główny wątek obsługuje zapytania HTTP.

### 6.3.1 Data Plane: Serwer TCP (AsyncIO)

Obsługa strumieniowania audio realizowana jest przez moduł `backend_async.py`. Zamiast klasycznego modelu HTTP (Request-Response), zastosowano stałe połączenie TCP, co minimalizuje opóźnienia.

Przebieg sesji komunikacyjnej:

1. **Handshake i Autoryzacja:** Po nawiązaniu połączenia, serwer oczekuje na 8-bajtowy identyfikator urządzenia oraz zaszyfrowany nagłówek autoryzacyjny. Weryfikacja nagłówka (funkcja `check_header`) obejmuje sprawdzenie poprawności deszyfracji oraz walidację znacznika czasu (Timestamp), co chroni system przed atakami typu Replay.
2. **Odbiór Audio (TLV):** Dane są odbierane w pętli zgodnie ze schematem *Type-Length-Value*. Serwer najpierw czyta 4 bajty nagłówka (rozmiar paczki), a następnie dokładnie tyle bajtów zaszyfrowanego ładunku. Odebranie bajtu o wartości `0x00` lub zamknięcie gniazda sygnalizuje koniec transmisji.

3. **Przetwarzanie:** Pakiety są deszyfrowane w locie (kluczem pobranym z bazy dla danego Device ID) i sklejane w buforze pamięci, a następnie zapisywane jako plik WAV.
4. **Odpowiedź Zwrotna:** Po wygenerowaniu odpowiedzi przez AI, serwer wykorzystuje to samo, wciąż otwarte połączenie TCP do wysłania strumienia zwrotnego. Wysyłka poprzedzona jest bajtem sterującym 0x01.

### 6.3.2 Control Plane: Serwis MQTT

Moduł `mqtt_service.py` odpowiada za lekką komunikację sterującą. Zaimplementowano tu unikalny mechanizm parowania (Provisioning):

- Gdy serwer otrzyma komunikat na kanale `devices/provisioning`, nie wie, od jakiego urządzenia on pochodzi (brak ID w temacie).
- System pobiera z bazy listę wszystkich urządzeń o statusie `pending` i iteracyjnie próbuje odszyfrować wiadomość przy użyciu klucza AES każdego z nich.
- Udana deszyfracja jednoznacznie identyfikuje urządzenie, co kończy proces parowania (zmiana statusu na `paired`).

### 6.3.3 Web Serwer i API (FastAPI)

Moduł `webapp.py` udostępnia interfejsy dla użytkownika końcowego:

- **Dashboard WWW:** Renderowany po stronie serwera (SSR) przy użyciu silnika szablonów Jinja2. Umożliwia logowanie, przeglądanie logów i konfigurację urządzeń. Sesja utrzymywana jest w bezpiecznych ciasteczkach (HttpOnly Cookies) zawierających token JWT.
- **REST API dla Mobile:** Zestaw endpointów JSON (np. `/api/device/register`) zabezpieczonych tokenem Bearer JWT. Służą one aplikacji mobilnej do rejestracji nowych urządzeń (przekazanie klucza AES wygenerowanego przez telefon) oraz sprawdzania statusu połączenia.

## 6.4 Potok przetwarzania AI (AI Pipeline)

Logika generowania odpowiedzi (funkcja `process_audio`) realizowana jest sekwencyjnie:

1. **Transkrypcja (STT):** Plik audio użytkownika przesyłany jest do API **OpenAI Whisper**, które zwraca transkrypcję tekstową.
2. **Generowanie Tekstu (LLM):** Treść zapytania łączona jest z `system_prompt` pobranym z bazy danych i przesyłana do modelu **GPT-4o-mini**.
3. **Synteza Mowy (TTS):** Tekst odpowiedzi trafia do API **ElevenLabs**. Otrzymany plik MP3 jest konwertowany biblioteką `pydub` do formatu natywnego dla ESP32: **WAV, 16000 Hz, Mono, 16-bit**. Zmiana próbkowania z domyślnych 44.1kHz na 16kHz (standard Wideband Audio) pozwoliła na znaczną redukcję objętości danych przy zachowaniu wysokiej czytelności mowy.

## 6.5 Bezpieczeństwo danych

Backend implementuje spójną z firmwarem politykę bezpieczeństwa ("Security by Design"):

- **Szyfrowanie End-to-End:** Wszystkie dane audio oraz konfiguracyjne przesyłane przez sieć (TCP i MQTT) są szyfrowane algorytmem AES-256 GCM. Backend weryfikuje tag uwierzytelniający (Auth Tag) każdego pakietu.
- **Zarządzanie kluczami:** Klucze szyfrujące nie są wpisane na stałe w kodzie. Są generowane dynamicznie przez aplikację mobilną, przesyłane bezpiecznym kanałem HTTPS do bazy danych serwera i nigdy nie opuszczają bezpiecznego środowiska backendu.
- **Autoryzacja:** Dostęp do panelu administracyjnego i API wymaga ważnego tokenu JWT (JSON Web Token), podpisanego tajnym kluczem serwera (`SECRET_KEY`). Tokeny mają ograniczony czas ważności (60 minut), co minimalizuje ryzyko w przypadku ich przechwycenia.

# Rozdział 7

## Implementacja Aplikacji Mobilnej

Uzupełnieniem ekosystemu „ESPhera” jest dedykowana aplikacja mobilna, pełniąca rolę centrum sterowania i konfiguracji. Aplikacja została zrealizowana w technologii wielopłatformowej, co pozwala na jej uruchomienie zarówno na systemie Android, jak i iOS przy użyciu jednego kodu źródłowego.

### 7.1 Technologie i struktura projektu

Aplikację napisano w języku Dart z wykorzystaniem frameworka Flutter (wersja SDK  $\geq 3.10$ ). Wybór ten podyktowany był możliwością szybkiego prototypowania interfejsów użytkownika (UI) oraz dostępnością rozbudowanych bibliotek do obsługi komunikacji Bluetooth Low Energy.

Kluczowe biblioteki wykorzystane w projekcie (plik `pubspec.yaml`):

- **flutter\_blue\_plus**: Obsługa skanowania, łączenia i wymiany danych z urządzeniami BLE.
- **encrypt**: Implementacja algorytmów kryptograficznych (AES) po stronie klienta mobilnego.
- **http**: Komunikacja z REST API serwera backendowego.
- **flutter\_secure\_storage**: Bezpieczne przechowywanie tokenów JWT (JSON Web Token) w pęku kluczy systemu operacyjnego (Keystore/Keychain).
- **provider**: Zarządzanie stanem aplikacji.

### 7.2 Proces parowania (Provisioning BLE)

Najbardziej złożonym modułem aplikacji jest ekran parowania (`pair_device_page.dart`), który realizuje proces bezpiecznego przekazania danych sieci WiFi do mikrokontrolera ESP32. Proces ten przebiega według następującego algorytmu:

1. **Skanowanie**: Aplikacja filtruje urządzenia BLE, poszukując tych, których nazwa rozpoczyna się od frazy „ESPhera”.
2. **Połączenie GATT**: Po wybraniu urządzenia, następuje połączenie z serwerem GATT i odkrycie serwisu o UUID `0xA001`.

3. **Generacja kluczy:** Aplikacja generuje losowy, 128-bitowy klucz sesji (`_generatedDeviceKey`). Jest to klucz, który posłuży do szyfrowania danych WiFi oraz późniejszej komunikacji urządzenia z chmurą.
4. **Handshake (Wymiana kluczy):** Klucz sesji jest szyfrowany przy użyciu zaszyfowanego w aplikacji „Klucza Fabrycznego” (`ESHERA_PROD_KEY`) i wysyłany na charakterystykę `Custom Key` (UUID `0xA014`). Dzięki temu, tylko oryginalne urządzenie ESPhera jest w stanie odszyfrować ten klucz.
5. **Konfiguracja WiFi:** Nazwa sieci (SSID) oraz hasło wprowadzone przez użytkownika są szyfrowane nowym kluczem sesji i wysyłane na odpowiednie charakterystyki BLE.
6. **Rejestracja w chmurze:** Równolegle, aplikacja mobilna wysyła wygenerowany klucz sesji do serwera backendowego poprzez zapytanie HTTP POST na endpoint `/device/register`. Dzięki temu serwer (Python) będzie w stanie deszyfrować przyszłe strumienie audio pochodzące z tego konkretnego urządzenia.

## 7.3 Konfiguracja parametrów Asystenta

Aplikacja umożliwia personalizację zachowania asystenta poprzez panel konfiguracyjny (`device_config_page.dart`). Użytkownik ma możliwość modyfikacji parametrów przechowywanych w bazie danych serwera.

Dostępne opcje konfiguracji:

- **System Prompt:** Pole tekstowe pozwalające zdefiniować „osobowość” asystenta (np. „Jesteś sarkastycznym asystentem”, „Odpowiadaj krótko i zwięźle”). Zmiana tego pola wpływa bezpośrednio na kontekst przesyłany do modelu GPT-4o-mini.
- **Model Głosu (Sound Model):** Lista rozwijana pozwalająca wybrać jeden z dostępnych głosów ElevenLabs (np. Brian, Alice, Bill).
- **Jasność LED:** Suwak regulujący natężenie światła pierścienia LED w stanie spoczynku.
- **Sleep Timeout:** Czas bezczynności, po którym urządzenie przechodzi w tryb głębokiego uśpienia.

Komunikacja w tym module odbywa się wyłącznie przez protokół HTTPS (metody GET do pobrania obecnej konfiguracji oraz POST do zapisu zmian), autoryzowaną tokenem JWT użytkownika.

## 7.4 Interfejs Użytkownika (UI)

Interfejs został zaprojektowany zgodnie z wytycznymi Material Design. Zastosowano responsywne widoki, które dostosowują się do wielkości ekranu urządzenia mobilnego. Aplikacja obsługuje również informowanie użytkownika o stanie połączenia poprzez komunikaty typu *SnackBar* (np. „WiFi Połączone!”, „Błąd rejestracji”).

# Rozdział 8

## Podsumowanie i wnioski

Celem niniejszej pracy było zaprojektowanie i realizacja inteligentnego asystenta głosowego „ESPhera”, opartego na mikrokontrolerze ESP32 oraz chmurze obliczeniowej. Zrealizowany prototyp łączy w sobie zagadnienia z obszaru systemów wbudowanych, inżynierii oprogramowania oraz sztucznej inteligencji.

### 8.1 Stopień realizacji celu

Wszystkie założone cele projektowe zostały zrealizowane. Powstał w pełni funkcjonalny system, który umożliwia:

- Dwukierunkową komunikację głosową z użytkownikiem w języku naturalnym.
- Bezpieczne podłączenie do sieci WiFi poprzez mechanizm Provisioningu przez Bluetooth LE.
- Szyfrowanie transmisji danych algorytmem AES-256 GCM, co zapewnia prywatność rozmów.
- Wizualizację stanu urządzenia za pomocą diod LED oraz wyświetlacza OLED, obsługiwanego przez autorski sterownik.

Szczególnym osiągnięciem jest implementacja oprogramowania układowego (Firmware) w nowoczesnym standardzie C++20, z wykorzystaniem modułów oraz zaawansowanych mechanizmów abstrakcji.

### 8.2 Wnioski końcowe

Realizacja projektu pozwoliła na sformułowanie następujących wniosków:

1. **Potencjał ESP32 w aplikacjach Audio/AI:** Mikrokontroler ESP32 dysponuje wystarczającą mocą obliczeniową do obsługi szyfrowania AES oraz strumieniowania audio w czasie rzeczywistym, pod warunkiem wykorzystania mechanizmów DMA oraz wielordzeniowości (rozdzielenie obsługi sieci i peryferiów).
2. **Przewaga C++20 nad C w Embedded:** Zastosowanie modułów C++ znacząco poprawiło strukturę projektu i czytelność kodu, choć wymagało to przełamania barier związanych z konfiguracją narzędzi (toolchain) w ekosystemie ESP-IDF.



3. **Opóźnienia a User Experience:** Czas reakcji systemu jest akceptowalny dla asystenta domowego, jednak jest on ściśle uzależniony od jakości połączenia internetowego oraz czasu odpowiedzi zewnętrznych API. Przeniesienie części logiki (np. prostych komend) bezpośrednio na urządzenie (Edge AI) mogłoby znacząco skrócić ten czas.
4. **Własne sterowniki sprzętowe:** Rezygnacja z gotowych bibliotek (np. dla ekranu OLED) pozwoliła na głębsze zrozumienie protokołów komunikacyjnych (I2C) i optymalizację zużycia pamięci, co jest kluczowe w systemach o ograniczonych zasobach.